

Specyfikacja Funkcjonalna Systemu Wizualizacji Grafów – Część JAVA

Mikołaj Dulny
Maciej Maciejewski

9 czerwca 2026

Spis treści

1	Wstęp i Cel Projektu	3
2	Wypis Funkcjonalności Programu	3
3	Opis Interfejsu i Instrukcja Obsługi	3
3.1	Pasek Menu	4
3.2	Główny Panel Wizualizacji (Płótno)	4
3.3	Panel Narzędziowy (Prawy pasek)	4
4	Formaty Danych (Wejście i Wyjście)	5
4.1	Dane wejściowe	5
4.2	Dane wyjściowe	5
5	Sytuacje Wyjątkowe i Komunikaty Błędów	5

1 Wstęp i Cel Projektu

Niniejszy dokument stanowi specyfikację funkcjonalną aplikacji z graficznym interfejsem użytkownika, służącej do wizualizacji grafów planarnych. Aplikacja jest przeznaczona dla użytkowników końcowych i współpracuje z danymi wygenerowanymi przez moduł obliczeniowy (napisanym w języku C). Celem programu jest przejrzyste i estetyczne wyświetlenie wyznaczonych współrzędnych wierzchołków oraz umożliwienie użytkownikowi interakcji z wygenerowanym grafem.

2 Wypis Funkcjonalności Programu

Zgodnie z założeniami projektowymi, aplikacja udostępnia użytkownikowi następujące funkcjonalności:

- **Integracja z silnikiem obliczeniowym C:** Bezpośrednie uruchamianie modułu C z poziomu interfejsu Javy za pomocą środowiska WSL, wraz z przekazywaniem plików i odbieraniem wyników w czasie rzeczywistym.
- **Wczytywanie danych:** Obsługa plików wejściowych zawierających strukturę grafu oraz współrzędne, zapisanych w formacie tekstowym (.txt) lub binarnym (.bin).
- **Wizualizacja grafu:** Automatyczne rysowanie układu wierzchołków i łączących je krawędzi (wraz z ich wagami) na dwuwymiarowej płaszczyźnie.
- **Wybór algorytmu:** Możliwość przeliczania lub dostosowywania układu grafu na podstawie wybranego algorytmu wizualizacji.
- **Interakcja z widokiem (Nawigacja):** Przybliżanie i oddalanie widoku (zoom), przesuwanie pola widzenia oraz powrót do widoku domyślnego.
- **Zarządzanie wyświetlaniem:** Opcja pokazywania lub ukrywania etykiet identyfikujących wierzchołki oraz wag przypisanych do krawędzi.
- **Ręczna modyfikacja:** Edycja współrzędnych (X, Y) dla wskazanego przez użytkownika wierzchołka, z natychmiastowym odświeżeniem widoku.
- **Eksport wyników:** Zapis zmodyfikowanych współrzędnych do nowego pliku tekstowego oraz możliwość zapisu bieżącego widoku grafu w formie pliku graficznego.

3 Opis Interfejsu i Instrukcja Obsługi

Interfejs programu został zaprojektowany z myślą o intuicyjnej obsłudze. Składa się z trzech głównych sekcji:

3.1 Pasek Menu

Znajduje się w górnej części okna i zawiera opcje operacji na plikach:

- **Plik → Wczytaj graf:** Otwiera systemowe okno wyboru pliku, pozwalając załadować graf z formatu tekstowego lub binarnego.
- **Plik → Zapisz współrzędne:** Pozwala zapisać zaktualizowaną listę współrzędnych do wybranego pliku.
- **Plik → Eksportuj widok:** Tworzy obraz (zrzut widocznego grafu) możliwy do zapisu na dysku.

3.2 Główny Panel Wizualizacji (Płótno)

Centralna część aplikacji, w której rysowany jest graf. Użytkownik może wchodzić w interakcję z widokiem przy użyciu myszy:

- Kliknięcie wierzchołka powoduje jego zaznaczenie (co aktywuje opcję jego modyfikacji w panelu bocznym).
- Użycie rolki myszy pozwala na skalowanie widoku (zoom).

3.3 Panel Narzędziowy (Prawy pasek)

Panel ułatwiający sterowanie widokiem i modyfikację parametrów:

- **Uruchomienie obliczeń:** Przycisk "Przelicz graf (Silnik C)". Jego kliknięcie powoduje wyświetlenie okna instruktażowego, a następnie poproszenie użytkownika o wskazanie pliku wejściowego (np. z definicją krawędzi), który zostaje automatycznie przekazany do silnika.
- **Wybór algorytmu:** Lista rozwijana pozwalająca zmienić metodę układania wierzchołków dla modułu C.
- **Modyfikacja wierzchołka:** Pola numeryczne "X" i "Y" umożliwiające ręczną zmianę pozycji zaznaczonego wierzchołka.
- **Opcje wyświetlania (Filtry):** Zestaw przełączników (checkbox) włączających i wyłączających widoczność ID wierzchołków oraz wag krawędzi.
- **Kontrola widoku:** Suwak zbliżenia (zoom) oraz przycisk "Wyśrodkuj graf", który dynamicznie oblicza środek ciężkości obecnego układu wierzchołków i przesuwa cały graf na środek widocznego płótna roboczego.

4 Formaty Danych (Wejście i Wyjście)

Program do poprawnego działania wymaga dostarczenia plików o ściśle określonej strukturze, wynikającej z działania wcześniejszego modułu (C).

4.1 Dane wejściowe

Użytkownik może wczytać plik w dwóch formatach:

- **Plik tekstowy (.txt):** Wymagane jest, aby dane były zapisane w postaci listy. Spodziewany format to np. linijki zawierające identyfikator wierzchołka oraz jego współrzędne `<wierzchołek> <x> <y>`.
- **Plik binarny (.bin):** Alternatywny format przyspieszający ładowanie bardzo dużych grafów, stanowiący binarny zapis struktury wygenerowanej z programu C.

4.2 Dane wyjściowe

Wynikiem działania programu, w przypadku chęci zapisu, jest plik tekstowy. Będzie on zawierał zaktualizowane (na skutek użycia algorytmu lub ręcznej modyfikacji) pozycje wierzchołków w analogicznym formacie `<ID> <X> <Y>`.

5 Sytuacje Wyjątkowe i Komunikaty Błędów

Aplikacja została zabezpieczona przed podstawowymi błędami ze strony użytkownika oraz systemu. Wystąpienie błędu będzie sygnalizowane wyraźnym okienkiem dialogowym z ostrzeżeniem.

- **Błąd odczytu pliku:** Jeśli wybrany plik nie istnieje, jest uszkodzony lub ma nieprawidłowe uprawnienia, program wyświetli komunikat.
- **Błąd integracji z silnikiem C:** W przypadku braku dostępu do środowiska WSL, braku pliku wykonywalnego modułu C lub błędów obliczeniowych, Java przechwyci błąd i wyświetli go użytkownikowi w czytelnym oknie dialogowym, chroniąc interfejs przed zawieszeniem.
- **Błędne wartości w panelu modyfikacji:** Jeśli po zaznaczeniu wierzchołka użytkownik wprowadzi do pól edycji X/Y znaki inne niż liczby zmiennoprzecinkowe, próba zatwierdzenia zmiany wywoła komunikat. Zmiany nie zostaną zaaplikowane.
- **Współrzędne poza obszarem roboczym:** Próba wpisania X lub Y wykraczającego poza widoczny ekran zostanie zablokowana specjalnym komunikatem.

- **Niezgodność formatu danych:** Program rygorystycznie weryfikuje strukturę każdego wiersza. Wymaga podania dokładnie 3 parametrów dla definicji wierzchołka oraz 4 dla krawędzi. Aplikacja przechwytuje wyjątek i wyświetla okno dialogowe ze szczegółowym komunikatem błędu.

Dokumentacja Implementacyjna Systemu Wizualizacji Grafów – Część JAVA

Mikołaj Dulny
Maciej Maciejewski

9 czerwca 2026

Spis treści

1	Wstęp	3
2	Architektura Systemu i Wzorce Projektowe	3
3	Struktura Danych (Model)	3
3.1	Klasa Vertex	3
3.2	Klasa Edge	4
3.3	Klasa GraphModel	4
4	Interfejs Graficzny (View)	4
4.1	Klasa MainWindow	4
4.2	Klasa GraphCanvas	4
4.3	Klasa ControlPanel	5
5	Logika Sterująca (Controller)	5
5.1	Klasa GraphController	5
6	Przetwarzanie Danych Wejściowych (Parser)	5
6.1	Klasa GraphParser	6
6.2	Parsowanie strumienia konsoli (Regex)	6

1 Wstęp

Niniejszy dokument stanowi dokumentację implementacyjną dla aplikacji okienkowej napisanej w języku Java, z wykorzystaniem biblioteki Swing. Aplikacja stanowi drugą część projektu i służy do wizualizacji grafów planarnych na podstawie danych wygenerowanych przez moduł napisany w języku C. Dokument pełni rolę przewodnika po kodzie, ułatwiającego jego zrozumienie, pielęgnację oraz ewentualny dalszy rozwój.

2 Architektura Systemu i Wzorce Projektowe

Głównym wzorcem projektowym zastosowanym w aplikacji jest MVC (**Model-View-Controller**). Wzorzec ten pozwala na logiczne i fizyczne oddzielenie warstwy przechowującej dane od interfejsu graficznego oraz logiki sterującej operacjami. Aby zachować przejrzystość kodu, system został podzielony na następujące pakiety:

- **model** – pakiet zawierający klasy reprezentujące strukturę grafu oraz operacje na plikach.
- **view** – pakiet grupujący klasy interfejsu graficznego (GUI), zbudowane w oparciu o komponenty biblioteki Swing.
- **controller** – pakiet zawierający logikę sterującą, łączącą zdarzenia z interfejsu z modelem danych.

3 Struktura Danych (Model)

Fundamentem aplikacji jest pakiet modelu, który przechowuje abstrakcyjną reprezentację grafu, niezależną od widoku.

3.1 Klasa Vertex

Klasa reprezentująca pojedynczy wierzchołek grafu. Przechowuje ona podstawowe parametry fizyczne wyliczone przez moduł C.

- **Pola klasy:** `id` (unikalny identyfikator), `x` (współrzędna pozioma), `y` (współrzędna pionowa).
- **Kluczowe metody:** Gettery i settery pozwalające na modyfikację współrzędnych (np. przy ręcznym przesuwaniu wierzchołka).

3.2 Klasa Edge

Klasa reprezentująca pojedynczą krawędź grafu, łączącą dwa punkty.

- **Pola klasy:** Referencje do dwóch połączonych obiektów `Vertex` oraz zmienna `weight` (waga krawędzi).
- **Kluczowe metody:** Gettery zwracające połączone wierzchołki. Klasa z założenia jest niemutowalna po zainicjalizowaniu.

3.3 Klasa GraphModel

Klasa pełniąca rolę głównego kontenera danych w architekturze MVC. Agreguje ona wszystkie obiekty wierzchołków i krawędzi.

- **Pola klasy:** Listy dynamiczne `List<Vertex>` oraz `List<Edge>`.
- **Kluczowe metody:** Metody modyfikujące strukturę (np. `addVertex`, `clear`) wykorzystywane podczas ładowania nowych danych.

4 Interfejs Graficzny (View)

Pakiet widoku jest odpowiedzialny wyłącznie za renderowanie danych oraz wystawianie komponentów, z którymi użytkownik może wchodzić w interakcje.

4.1 Klasa MainWindow

Główna ramka aplikacji dziedzicząca po klasie `JFrame`. Jej zadaniem jest organizacja układu przestrzennego (np. za pomocą `BorderLayout`) oraz integracja paska menu, płótna do rysowania i panelu bocznego w spójną całość. Zaimplementowany pasek menu umożliwia wczytywanie plików, zapis zmienionych współrzędnych (`.txt`) oraz bezpośredni eksport aktualnego widoku grafu do pliku graficznego (`.png`).

4.2 Klasa GraphCanvas

Autorski komponent rysujący, dziedziczący po klasie `JPanel`.

- **Renderowanie:** Nadpisuje metodę `paintComponent(Graphics g)`. Iteruje po listach krawędzi i wierzchołków pobranych z obiektu `GraphModel`, używając narzędzi z pakietu `java.awt.Graphics2D` do rysowania linii i okręgów.
- **Transformacje:** Implementuje logikę przeliczania współrzędnych modelu (wygenerowanych przez program C) na współrzędne pikselowe ekranu, uwzględniając aktualny współczynnik skalowania (zoom) oraz przesunięcie pola widzenia.

- **Mechanizm wyśrodkowania:** Klasa zawiera metodę `centerGraph()`, która wyznacza obszar graniczny grafu (Bounding Box) wyszukując skrajne współrzędne X oraz Y. Następnie oblicza wektor przesunięcia względem aktualnego środka płótna (uwzględniając zoom) i aplikuje translację na wszystkich obiektach `Vertex`.

4.3 Klasa `ControlPanel`

Panel narzędziowy (`JPanel`) umieszczony z boku ekranu. Zawiera instancje takich komponentów jak `JComboBox` (do wyboru algorytmu), `JTextField` (do edycji współrzędnych) oraz `JCheckBox` (do sterowania filtrami wyświetlania). Układ komponentów zarządzany jest przez `BoxLayout`. Przycisk zapisu współrzędnych implementuje dynamiczną walidację granic okna w czasie rzeczywistym – wykorzystuje metody `getWidth()` oraz `getHeight()` obiektu `GraphCanvas`, co skutecznie zapobiega przesunięciu wierzchołków poza renderowany obszar. Z poziomu tego panelu uruchamiany jest również bezpośrednio moduł obliczeniowy C w środowisku WSL przy użyciu `ProcessBuilder`, co eliminuje konieczność ręcznego obsługiwania konsoli.

5 Logika Sterująca (Controller)

Zadaniem klas z tego pakietu jest reagowanie na akcje użytkownika. Obiekty kontrolera implementują interfejsy nasłuchiwczy z biblioteki Swing.

5.1 Klasa `GraphController`

Główny mózg operacji. Podpina `ActionListener` do przycisków w `ControlPanel` oraz `MouseListener` do `GraphCanvas`.

- **Obsługa zdarzeń myszy:** Po kliknięciu na płótnie, kontroler oblicza, czy w danym punkcie znajduje się wierzchołek. Jeśli tak, pobiera jego parametry i przekazuje do pól tekstowych w panelu bocznym.
- **Synchronizacja:** Każda zmiana stanu w modelu (np. po wczytaniu nowego pliku lub ręcznej edycji współrzędnych) kończy się wywołaniem metody `repaint()` na obiekcie widoku, co wymusza odświeżenie interfejsu.

6 Przetwarzanie Danych Wejściowych (Parser)

Aby zapewnić poprawną komunikację między modulem napisanym w języku C a aplikacją w języku Java, zaimplementowano dedykowany moduł parsujący.

6.1 Klasa GraphParser

Odpowiada za wejście/wyjście (I/O). Jej zadaniem jest odczyt plików wynikowych dostarczonych przez program w C i przetworzenie ich na obiekty Javy.

- **Odczyt formatu .txt:** Używa klas `BufferedReader` i `Scanner` do sekwencyjnego czytania linii, rozdzielania ich po znakach spacji i konwersji ciągów znaków na typy liczbowe `Integer` oraz `Double`.
- **Odczyt formatu .bin:** Używa bezpośredniego wczytywania bajtów, co przyspiesza proces ładowania dużych struktur grafowych z uwzględnieniem zapisu Little-Endian pochodzącego z C.
- **Walidacja:** Moduł obsługuje wyjątki `IOException` oraz `NumberFormatException`, aby w razie uszkodzonego pliku przerwać parsowanie i zgłosić błąd.

6.2 Parsowanie strumienia konsoli (Regex)

Aby wyeliminować problem niejednoznaczności ścieżek wyjściowych na styku Windows-Linux (WSL), aplikacja odczytuje bezpośrednio standardowy strumień wyjścia (`stdout`) procesu wykonującego kod C. Wyłapywane przez klasę `Scanner` logi są na bieżąco analizowane przez silnik wyrażeń regularnych (`Pattern`). Do wyłuskania identyfikatorów i współrzędnych w locie zastosowano bezpieczny wzorzec:

```
Wierzcho[^:]+?\s+(\d+):\s+X=([\d.-]+),\s+Y=([\d.-]+)
```

Rozwiązanie to uodparnia program na niespójności z polskimi znakami diakrytycznymi w terminalu, umożliwiając natychmiastową aktualizację struktury `GraphModel` bez potrzeby korzystania z plików tymczasowych `.txt`.

Dokumentacja Końcowa Projektu Systemu Wizualizacji Grafów – Część JAVA

Mikołaj Dulny
Maciej Maciejewski

9 czerwca 2026

Spis treści

1	Podsumowanie i Cel Projektu	3
2	Realizacja Warstwy Wizualnej (GUI)	3
2.1	Autorski Moduł Renderujący	3
2.2	Interakcja i Edycja w Czasie Rzeczywistym	4
3	Nowatorskie Podejście do Integracji (Pipeline i Regex)	4
3.1	Sterowanie procesami potomnymi	4
3.2	Przetwarzanie strumieniowe „w locie”	4
3.3	Analiza Maszyną Stanów (Regex)	4
4	Zarządzanie Danymi (Wejście / Wyjście)	5
5	Wnioski i Perspektywy Rozwoju	5

1 Podsumowanie i Cel Projektu

Druga faza projektu obejmowała zaprojektowanie i implementację aplikacji okienkowej (typu desktop), której nadrzędnym zadaniem jest przejrzysta i interaktywna wizualizacja wielowymiarowych układów wierzchołków grafów planarnych. Aplikacja stanowi logiczne rozwinięcie i interfejs graficzny (frontend) dla silnika obliczeniowego napisanego w języku C.

Projekt został zrealizowany w oparciu o język Java z wykorzystaniem standardowej biblioteki graficznej Swing. Całość architektury oparto na sprawdzonym wzorcu projektowym Model-View-Controller (MVC), co zapewniło wysoką skalowalność kodu, ułatwiło testowanie poszczególnych komponentów oraz pozwoliło na całkowite odseparowanie logiki biznesowej od warstwy prezentacji.

Największym osiągnięciem projektowym było pokonanie bariery systemowej i stworzenie płynnego, wieloplatformowego pomostu komunikacyjnego. Pozwoliło to na bezproblemowe sterowanie silnikiem z poziomu systemu Windows, podczas gdy same obliczenia wykonywane są w izolowanym podsystemie Windows Subsystem for Linux (WSL).

2 Realizacja Warstwy Wizualnej (GUI)

Interfejs użytkownika został zaprojektowany z naciskiem na ergonomię i intuicyjność. W ramach prac nad modulem graficznym zrealizowano następujące kluczowe elementy:

2.1 Autorski Moduł Renderujący

Zrezygnowano z gotowych bibliotek do rysowania grafów na rzecz własnoręcznie zaimplementowanego płótna (`GraphCanvas`). Moduł ten, dziedziczący po `JPanel`, operuje bezpośrednio na zaawansowanym API `Graphics2D`.

- **Antialiasing:** Wdrożono sprzętowe wygładzanie krawędzi, co znacząco poprawiło estetykę i czytelność przecinających się linii oraz okręgów reprezentujących wierzchołki.
- **Dynamiczne centrowanie (Bounding Box):** Opracowano algorytm matematyczny, który na żądanie wyznacza obszar graniczny rysowanego grafu, wylicza jego środek ciężkości i wektorowo przesuwa cały układ współrzędnych tak, aby idealnie wypełniał on dostępne okno aplikacji.
- **Skalowanie (Zoom):** Dodano płynne skalowanie widoku w czasie rzeczywistym przy użyciu macierzy transformacji lub bezpośredniego przelicznika punktowego powiązanego z suwakiem w panelu bocznym.

2.2 Interakcja i Edycja w Czasie Rzeczywistym

Aplikacja przestała być jedynie statyczną przeglądarką plików, stając się pełnoprawnym edytorem.

- **Drag & Drop:** Wdrożono w pełni responsywną obsługę zdarzeń kursora (`MouseAdapter`). Użytkownik ma możliwość manualnego chwytania, przeciągania i upuszczania wybranych wierzchołków na płótnie, co pozwala na ręczne „rozplątywanie” skomplikowanych układów.
- **Bezpieczna edycja numeryczna:** Użytkownik może z precyzją co do ułamka zmieniać współrzędne w panelu bocznym. Pola tekstowe zostały zabezpieczone – wbudowana walidacja natychmiast wychwytyje błędy ludzkie (np. próby wpisania liter), blokuje wprowadzenie ich do modelu i informuje użytkownika o błędzie.

3 Nowatorskie Podejście do Integracji (Pipeline i Regex)

Fundamentalnym problemem podczas integracji modułu Javy (Windows) z modulem C (Linux/WSL) były konflikty w ścieżkach dostępu do plików oraz brak gwarancji poprawności zapisu na zamontowanych partycjach NT. Aby zniwelować te trudności, całkowicie zrezygnowano z klasycznego i uciążliwego mechanizmu szukania i podmieniania plików tymczasowych (`output.txt`).

3.1 Sterowanie procesami potomnymi

Wykorzystano zaawansowany obiekt biblioteki standardowej Javy – `ProcessBuilder`. Java inicjuje proces interpretera `bash`, po czym przekazuje mu odpowiednio sformatowane, bezwzględne polecenia wywołujące silnik C w docelowym folderze. Java całkowicie przejmuje rolę nadrzędnego kontrolera środowiska.

3.2 Przetwarzanie strumieniowe „w locie”

Zamiast obciążać dysk twardy operacjami I/O, aplikacja tworzy wirtualny rurociąg (ang. *pipeline*). Odbiera ona w czasie rzeczywistym standardowy strumień wyjścia (`stdout`) emitowany przez pracujący w tle program C.

3.3 Analiza Maszyną Stanów (Regex)

Każda linia logów przechwycona ze strumienia jest natychmiast weryfikowana przez silnik Wyrażeń Regularnych (`Regex`). Zastosowany wzorzec parsujący:

Wierzcho[~:]+?\s+(\d+):\s+X=(\d.-]+),\s+Y=(\d.-]+)

jest rozwiązaniem wysoce niezawodnym. Pozwoliło to na ominięcie problemów ze znakami diakrytycznymi (np. polskie znaki zmieniane przez konsolę na artefakty) oraz ignorowanie zbędnych białych znaków. Silnik wychwytuje wyłącznie to, co jest kluczowe z punktu widzenia modelu: identyfikator wierzchołka (grupa 1), pozycję X (grupa 2) oraz pozycję Y (grupa 3). Odczytane dane trafiają do pamięci w ułamku sekundy, oferując natychmiastowe odświeżenie ekranu, pozbawione jakichkolwiek opóźnień związanych z systemem plików.

4 Zarządzanie Danymi (Wejście / Wyjście)

Mimo stworzenia bezpośredniego połączenia obu środowisk, aplikacja w pełni zachowała zdolność do pracy jako niezależny byt.

- **Formaty plików:** Zaimplementowano odporne na uszkodzenia parsery potrafiące odtworzyć graf z tradycyjnych plików tekstowych, a także superszybki parser operujący na strumieniach bajtów, dedykowany plikom binarnym (.bin), z poszanowaniem architektury Little-Endian natywnej dla środowiska C.
- **Eksport wyników:** Zmodyfikowany graf może zostać na żądanie wyeksportowany do nowego pliku współrzędnych. Ponadto wprowadzono możliwość natychmiastowego zrzutu wizualnej reprezentacji płótna roboczego do bezstratnego formatu rastrowego .png, co znacząco ułatwia proces raportowania.

5 Wnioski i Perspektywy Rozwoju

Opracowany system w całości i z nadwyżką spełnia rygorystyczne założenia stawiane w dokumentacji początkowej. Aplikacja charakteryzuje się bardzo wysoką stabilnością działania, natychmiastową responsywnością i jest całkowicie bezpieczna od niespodziewanych wycieków pamięci oraz wyjątków typu `NullPointerException`.

Zastosowanie wzorców obiektowych oraz odseparowanie procesu parsowania od renderowania sprawia, że rozwiązanie to jest wysoce skalowalne. W przyszłości aplikacja może zostać w prosty sposób rozbudowana o obsługę grafów skierowanych, animację przebiegu algorytmów obliczeniowych (tzw. „krok po kroku”) lub implementację kolejnych metod rozkładu przestrzennego bez konieczności jakiegokolwiek ingerencji w dotychczasową warstwę graficzną.